Harness: оркестрация и API модулей

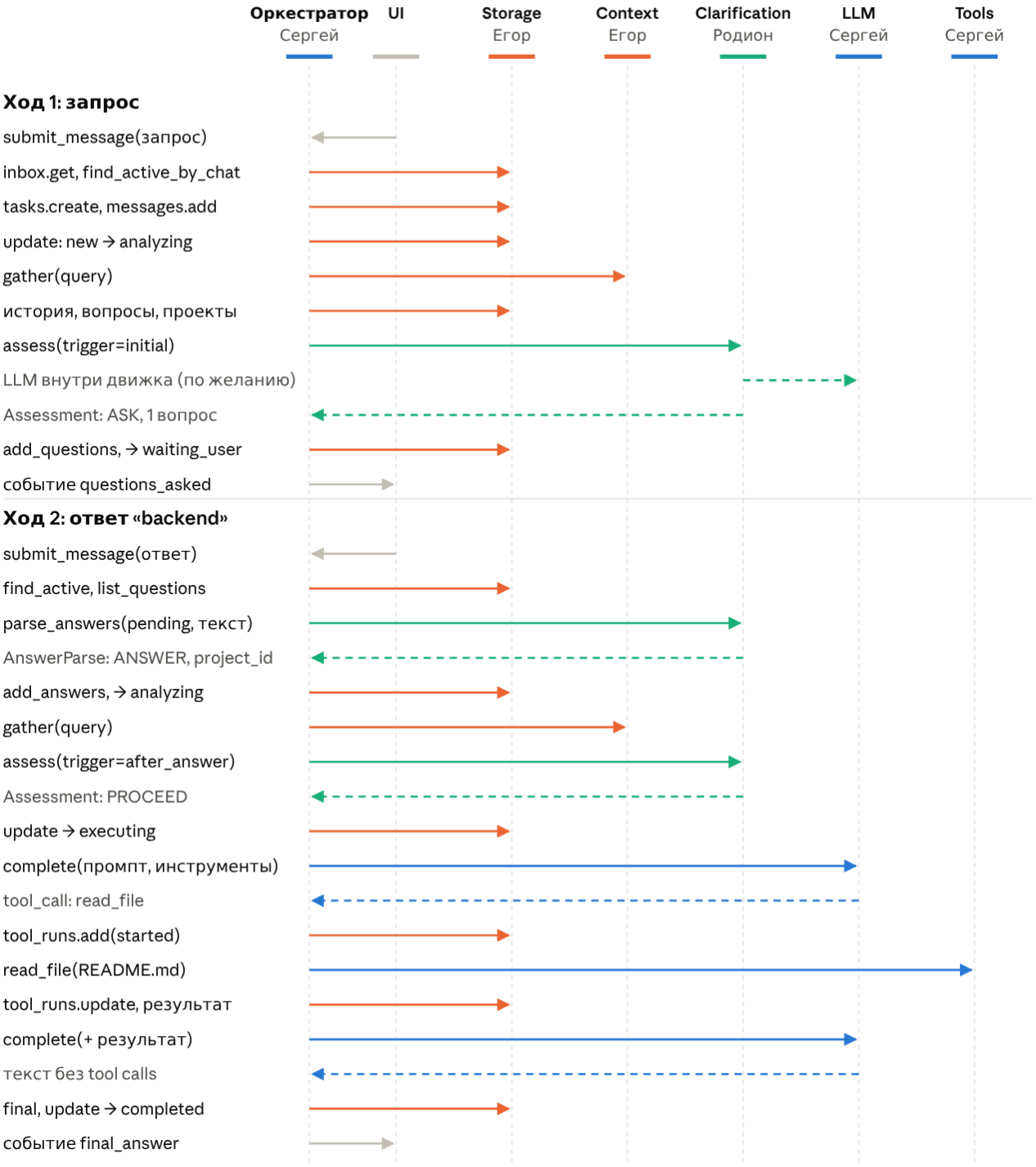
Nov 7, 2026 · @Sergey Kashkov

Схема оркестрации

Оркестратор — единственный модуль, который вызывает остальные и меняет статус задачи. Ниже три уровня детализации: кто кого вызывает в типичном сценарии, куда попадает входящее сообщение и что происходит внутри главного цикла. Цвет показывает, чей модуль вызывается: Егора (Storage, Context Engine), Родиона (Clarification Engine) или Сергея (LLM, инструменты).

Модули не вызывают друг друга — каждый вызов идёт от оркестратора

Сценарий «Проверь тесты проекта», в чате два проекта · пунктир — ответ на вызов

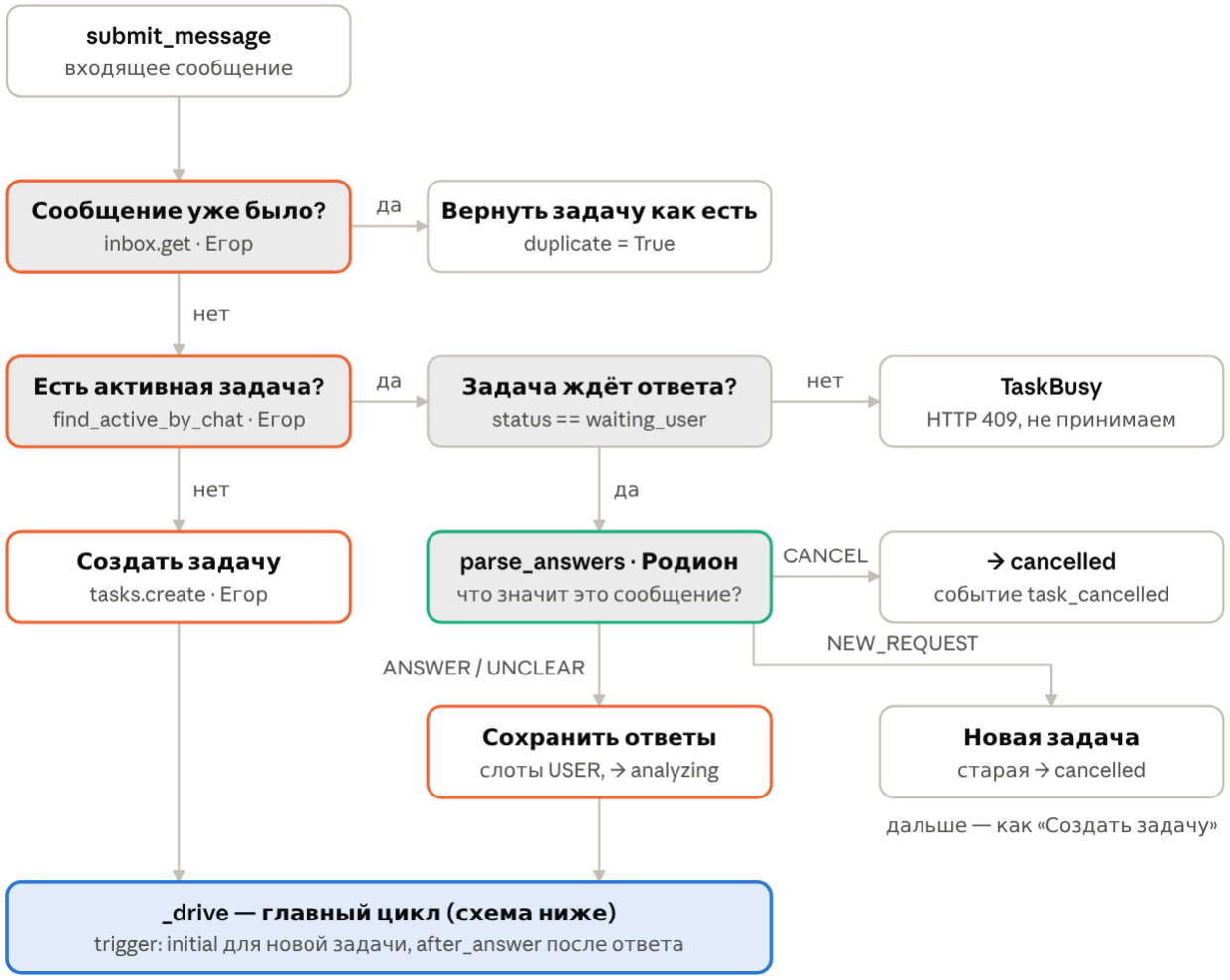


последовательность вызовов · два хода пользователя

Слева — что именно вызывается, стрелка ведёт к модулю-исполнителю. Каждая стрелка в Storage — отдельная транзакция; в executing вызовы LLM и инструментов повторяются, пока модель не вернёт текст без tool calls.

Сообщение в чате с ждущей задачей — это ответ, иначе новая задача

Залитые блоки — проверки; цвет рамки — чей модуль вызывается на этом шаге

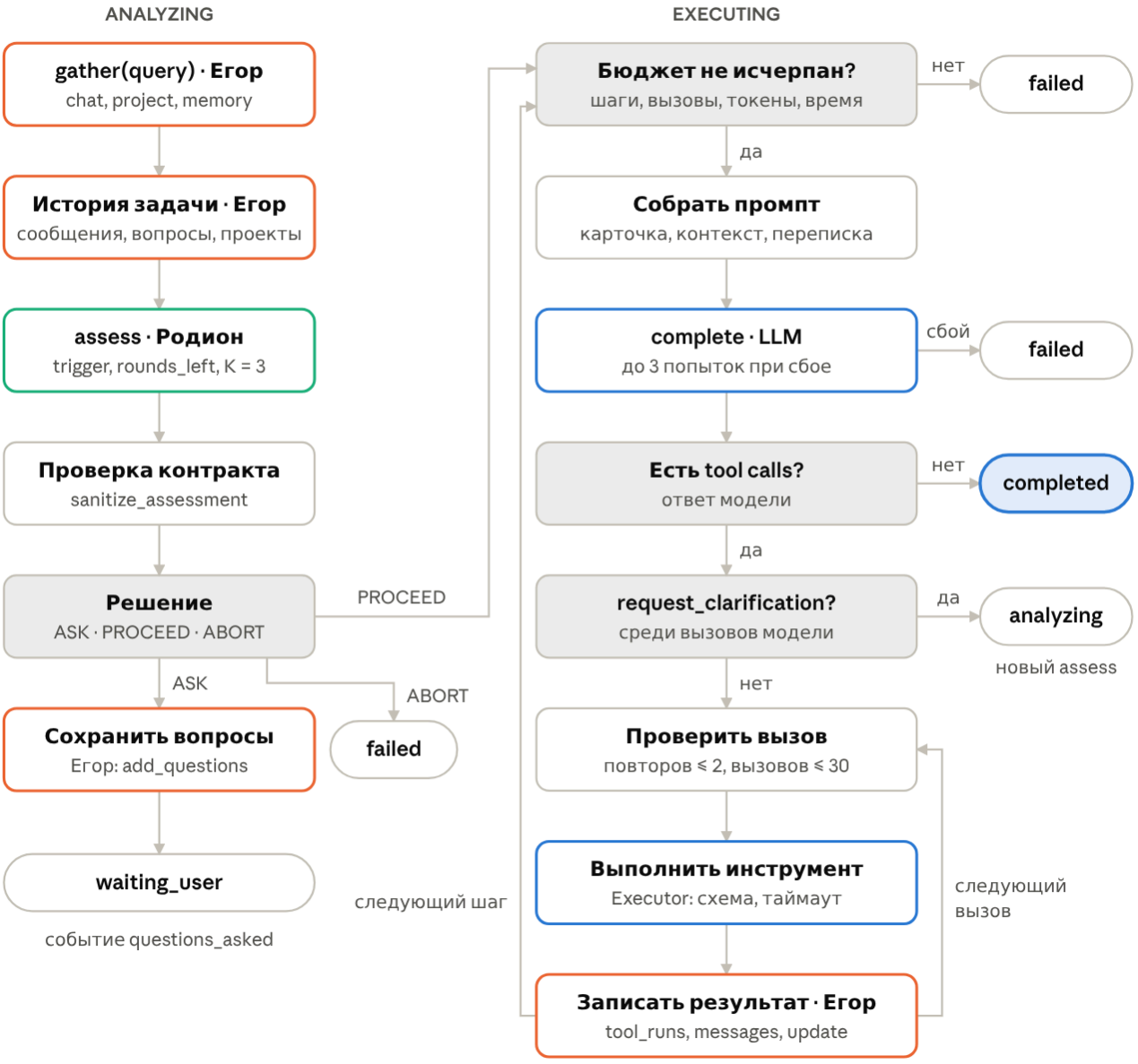


маршрутизация входящего сообщения · 3 проверки, 6 исходов

В MVP на чат допускается одна активная задача, поэтому смысл сообщения зависит только от статуса этой задачи. Разобрать, что именно написал пользователь — ответ, отмену или новую просьбу, — должен движок Родиона.

Выполнение начинается только после PROCEED, каждый шаг пишется в БД

Залитые блоки — проверки; цвет рамки — чей модуль; овалы — статус, в котором цикл останавливается



главный цикл _drive · анализ и выполнение

Цикл останавливается в одном из овалов: `waiting_user` ждёт ответа, `completed` и `failed` завершают задачу, `analyzing` начинает левую колонку заново с `trigger=execution_uncertainty`. Любое исключение в `gather` или `assess` ведёт в `failed(internal_error)`, а `cancel()` из другого запроса останавливает цикл на ближайшей записи в БД.

1. Зона ответственности и границы

Только оркестратор меняет статус задачи и решает, что делать дальше. Остальные модули — сервисы: они возвращают данные или решения, но сами состояние задачи не меняют. Благодаря этому каждый пишет свой модуль независимо, а интеграция сводится к соблюдению контрактов из `harness/contracts`.

Модуль	Владелец	Что предоставляет	Кто вызывает
Orchestrator: FSM, agent loop, сборка промпта, лимиты	Сергей	`submit_message`, `cancel`, `resume`, `get_task`	Interface
Interface: CLI, позже HTTP	Сергей (предложение)	тонкий слой над Orchestrator	пользователь
Tool Registry + Executor	Сергей (предложение)	`ToolRegistry`, `ToolExecutor.run`	Orchestrator
LLM-адаптер	Сергей (предложение)	`LLMClient.complete`	Orchestrator, Clarification Engine
Storage / Memory	Егор	`Storage.unit_of_work()` + репозитории	Orchestrator
Context Engine	Егор (предложение)	`ContextEngine.gather`	Orchestrator, инструменты
Clarification Engine	Родион	`assess`, `parse_answers`	Orchestrator

«Предложение» означает, что модуль явно ни за кем не закреплён. Это вынесено в открытые вопросы в конце документа.

Правила границ:

- Только оркестратор вызывает `uow.tasks.update` и меняет `tasks.status`.
- Clarification Engine не пишет в БД и не общается с пользователем. Он возвращает `Assessment`, а оркестратор проверяет инварианты и исполняет решение.
- Context Engine читает данные и регистрирует `context_blocks`. Связь блоков с задачей оркестратор сохраняет в `task.context_block_ids`.

- LLM вызывается только через `LLMClient`. Родион получает тот же клиент через конструктор своего движка, а оркестратор учитывает токены из `Assessment.usage`.
- Общие модели и протоколы лежат в `harness/contracts` и меняются только через PR с ревью всех троих.

2. Общие модели данных (контракт)

Все модули обмениваются Pydantic-моделями из `harness/contracts/models.py` (с `extra="forbid"`). Центральная идея — **TaskFrame**: набор «слотов» (параметров задачи), у каждого из которых известен источник. Именно через слоты Родион сообщает, что узнал, а оркестратор передаёт их в LLM.

```
class TaskStatus(StrEnum):
    NEW, ANALYZING, WAITING_USER, EXECUTING, COMPLETED, FAILED, CANCELLED

class SlotSource(StrEnum):        # приоритет при конфликте: USER > TOOL >
CONTEXT > ASSUMPTION
    USER, TOOL, CONTEXT, ASSUMPTION

class Slot:        name: str; value: Any; source: SlotSource; evidence:
list[str]; confidence: float
class Assumption:  slot_name: str; value: Any; rationale: str
class Gap:         id; slot_name; description; blocking: bool
                   information_gain; impact; user_cost; available_in_context
# 0..1, для ранжирования

class TaskFrame:   goal: str | None; slots: dict[str, Slot]; assumptions:
list[Assumption]; open_gaps: list[Gap]
class TaskCounters: clarification_rounds; questions_asked; agent_steps;
tool_calls; llm_calls; tokens_in; tokens_out
class FailureInfo: code: FailureCode; message: str

class Task:
    id; chat_id; user_id; project_id: str | None; request_text: str
    status: TaskStatus; frame: TaskFrame; counters: TaskCounters
    failure: FailureInfo | None; context_block_ids: list[str]
    version: int              # оптимистичная блокировка
    created_at; updated_at

class Message:     id; chat_id; task_id; role: user|assistant|tool|system
                   kind:
request|question|answer|tool_call|tool_result|final|error|note
                   content: str; tool_calls: list[ToolCall]; tool_call_id:
```

```
str | None

class ContextBlock:  id; sha256; source: ContextSource(kind, ref); text;
summary; score; tokens
class ContextQuery:  task_id; user_id; chat_id; project_id; text; scopes;
token_budget
class ContextBundle: blocks; total_tokens; truncated

class Question:    id; task_id; gap_id; slot_name; text; options: list[str];
priority; round; status
class Answer:      id; question_id; task_id; value: Any; raw_message_id

class ToolSpec:    name; description; parameters (JSON Schema); side_effects:
bool; timeout_s
class ToolCall:    id; name; arguments: dict
class ToolResult:  call_id; ok; content; error; duration_ms
class ToolRun:     id; task_id; step; call; status:
started|succeeded|failed|interrupted; result

class TaskSnapshot:  id; task_id; version; status; frame; counters;
transition(from, to, reason)
class OutboundEvent: kind:
questions_asked|progress|final_answer|task_failed|task_cancelled
                     task_id; chat_id; payload: dict
```

Договорённости:

- `Task` — единственный источник истины о текущем состоянии. `TaskSnapshot` — неизменяемая история версий для отладки и аудита, восстанавливаемся не из него.
- Рабочая переписка с LLM целиком хранится в `messages` задачи. Оркестратор не держит в памяти ничего, что нельзя восстановить из БД (кроме кеша контекста).
- Слот со статусом `USER` никто не перезаписывает слотом из контекста или допущением (это делает `TaskFrame.apply_slots`).
- Зарезервированный слот `project_id`: если он заполнен, оркестратор берёт проект из него, а не из `task.project_id`. Остальные имена слотов Родион выбирает сам.
- Все id — строки вида `task_<hex12>`, генерируются функцией `new_id(prefix)` на стороне создателя объекта, а не в БД. Так проще делать идемпотентность и тесты.
- Время везде в UTC (`datetime` с `tzinfo`).

3. API для Егора: Storage и Context Engine

Полный запрос к Егору — во вкладке `Запрос API: Егор`. Её можно отправить ему целиком: там описано, когда и какие методы Storage и Context Engine вызывает оркестратор и какие гарантии ему нужны.

4. API для Родиона: Clarification Engine

Полный запрос к Родиону — во вкладке `Запрос API: Родион`. Там описано, в каких ситуациях оркестратор вызывает `assess` и `parse_answers`, что передаёт, что ждёт на выходе и что делает с результатом.

5. Внешнее API оркестратора

Интерфейс видит четыре метода и поток событий. В MVP `submit_message` крутит цикл до ближайшей остановки (`waiting_user` или терминальный статус) и возвращает накопленные события. Этого достаточно для CLI и тестов. Для HTTP тот же код запускается в фоновой задаче, а события идут через SSE.

```
class Orchestrator:
    async def submit_message(self, msg: UserMessageIn) -> RunResult
    async def cancel(self, task_id: str) -> RunResult
    async def resume(self, task_id: str) -> RunResult
    async def get_task(self, task_id: str) -> Task

class UserMessageIn:  chat_id; user_id; text; project_id: str | None;
client_message_id: str | None
class RunResult:      task: Task; events: list[OutboundEvent]; duplicate:
bool
```

Маршрутизация входящего сообщения в `submit_message`:

1. `client_message_id` уже есть в `inbox` → вернуть текущее состояние задачи с `duplicate=True`, ничего не выполнять.
2. В чате есть задача в `waiting_user` → сообщение считается ответом, идёт в `parse_answers`.
3. В чате есть задача в `new`, `analyzing` или `executing` → `TaskBusy` (HTTP 409).
4. Иначе создаётся новая задача.

События для UI:

kind	Когда	payload
questions_asked	задача перешла в waiting_user	questions: [{id, text, options}], round
progress	выполнен вызов инструмента	tool, ok, step
final_answer	completed	text, assumptions: [{slot_name, value, rationale}]
task_failed	failed	code, message
task_cancelled	cancelled	reason

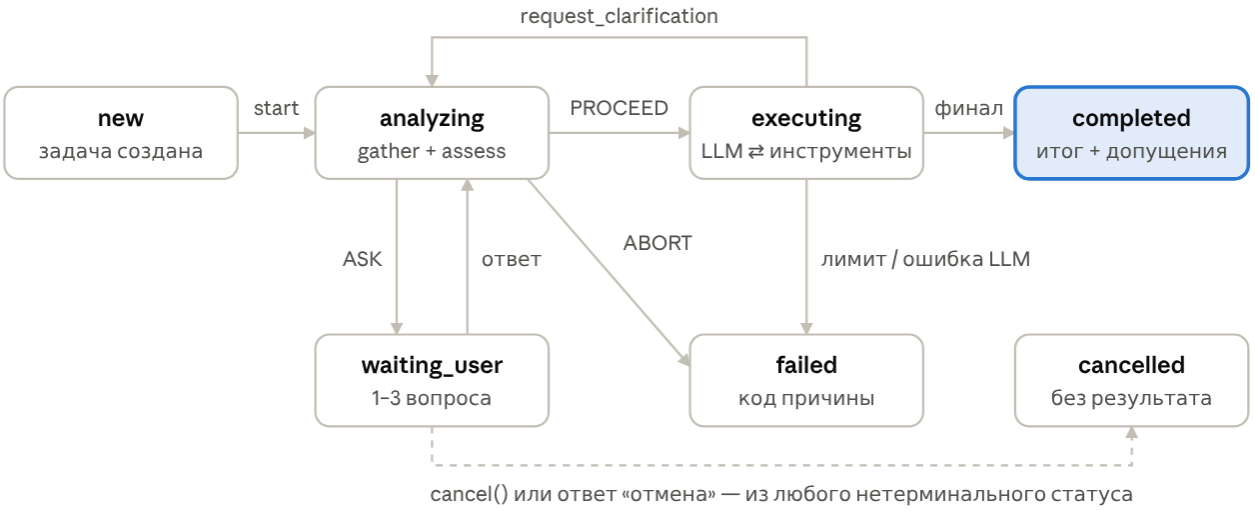
HTTP-обёртка (второй этап, FastAPI):

Метод	Путь	Вызывает
POST	/v1/chats/{chat_id}/messages	submit_message, заголовок Idempotency-Key = client_message_id
GET	/v1/tasks/{task_id}	get_task
POST	/v1/tasks/{task_id}/cancel	cancel
POST	/v1/tasks/{task_id}/resume	resume
GET	/v1/tasks/{task_id}/events	SSE-поток OutboundEvent

6. Конечный автомат задачи

У задачи семь статусов: шесть из MVP-документа плюс cancelled. Допустимые переходы заданы таблицей ALLOWED в harness/orchestrator/fsm.py. Функция transition(task, to, reason) бросает InvalidTransition на любой другой. Каждый переход сохраняется в той же транзакции, что и его побочные данные (вопросы, ответы, сообщения), и пишет TaskSnapshot.

До выполнения задача доходит только через проверку данных



статусы задачи · 7 статусов, 3 терминальных

Петля analyzing → waiting_user → analyzing — это цикл уточнения. Стрелка сверху — уточнение, которое модель запросила во время выполнения.

Из	Событие или условие	В	Действия оркестратора
—	новое сообщение без активной задачи	new	создать `Task`, сохранить `request`-сообщение, `inbox.put`
new	старт цикла	analyzing	—
analyzing	`assess` → ASK	waiting_user	сохранить вопросы и `question`-сообщение, `clarification_rounds += 1`, событие `questions_asked`
analyzing	`assess` → PROCEED	executing	записать слоты и допущения в frame
analyzing	`assess` → ABORT	failed	`clarification_exhausted` или `engine_abort`
waiting_user	ответ, intent ANSWER или UNCLEAR	analyzing	сохранить ответы, слоты `USER`, неотвеченные вопросы → `skipped`
waiting_user	ответ, intent CANCEL	cancelled	событие `task_cancelled`
waiting_user	ответ, intent NEW_REQUEST	cancelled	старая задача `superseded`, сообщение становится новой задачей
executing	LLM вернула текст без tool calls	completed	`final`-сообщение + блок допущений, событие `final_answer`

Из	Событие или условие	В	Действия оркестратора
executing	LLM вызвала `request_clarification`	analyzing	следующий `assess` с `trigger=execution_uncertainty`
executing	исчерпан бюджет или фатальная ошибка LLM	failed	`budget_exceeded` или `llm_error`
любой нетерминальный	`cancel(task_id)`	cancelled	CAS-запись, текущий шаг цикла получит `ConcurrencyError` и остановится
любой нетерминальный	исключение внутри чужого модуля	failed	`internal_error` с текстом исключения

Из `completed`, `failed`, `cancelled` переходов нет. Чтобы «перезапустить» задачу, создаётся новая.

Главный цикл `_drive` — диспетчер по статусу:

```
while not task.status.is_terminal and task.status != WAITING_USER:
    match task.status:
        case NEW:       task = await self._save_transition(task, ANALYZING,
"start")
        case ANALYZING: task = await self._analyze(task, trigger,
uncertainty, events)
        case EXECUTING: task, uncertainty = await self._execute_step(task,
events)
    # ConcurrencyError -> перечитать задачу; если она стала терминальной
(cancel) -> выход
```

7. Agent loop выполнения

Один шаг в `executing` — это один вызов LLM и все предложенные ею вызовы инструментов. Между шагами всё состояние лежит в БД. Поэтому цикл можно оборвать на любом шаге и продолжить через `resume`.

1. **Проверить бюджеты:** `agent_steps`, `tool_calls`, сумма токенов, срок текущего запуска. Превышение → `failed(budget_exceeded)`.
2. **Собрать промпт:** `PromptBuilder.build_execution(task, context, transcript)`, где transcript — `messages.list_by_task`.
3. **Вызвать LLM** с инструментами `registry.specs()` и псевдоинструментом `request_clarification`. Временные ошибки повторяются (раздел 9). Токены записываются в `counters`.

4. **Нет tool calls** → финальный ответ. Оркестратор сам добавляет блок «Допущения» из `frame.assumptions`, не полагаясь на LLM. Задача переходит в `completed`.
5. **Вызван `request_clarification(reason)`** → `analyzing`, дальше `assess` с `trigger=execution_uncertainty, uncertainty=reason`. Остальные tool calls этого ответа не выполняются: для них записывается результат-ошибка «отложено до уточнения».
6. **Иначе** — одна транзакция сохраняет `tool_call`-сообщение. Далее для каждого вызова:
   a. проверить повтор: тот же `(name, sha256(args))` уже выполнялся `max_repeated_tool_call` раз → вместо запуска вернуть ошибку «повторный вызов»;
   b. закоммитить `ToolRun(status=started)` — чтобы после падения было видно, что вызов начался;
   c. `ToolExecutor.run`: проверка аргументов по JSON Schema, таймаут, перехват исключений, обрезка вывода до `max_tool_output_chars`;
   d. закоммитить `ToolRun(succeeded|failed)` и `tool_result`-сообщение, событие `progress`.
7. `agent_steps += 1`, снапшот, переход к шагу 1.

Ошибка инструмента не валит задачу: она возвращается LLM как `ToolResult(ok=False, error=...)`, и модель решает, что делать дальше. От бесконечных повторов защищают лимиты шагов и повторных вызовов.

Инструменты MVP (`harness/tools/builtin.py`); все только читают данные, `side_effects=False`:

Инструмент	Аргументы	Что делает
`list_files`	`path="."`, `glob="*"`, `recursive=false`	список файлов в корне проекта
`read_file`	`path`, `max_bytes=20000`	текст файла
`search_files`	`pattern`, `glob="**/*"`, `max_results=50`	строки, совпавшие с regex, в формате `path:line: text`
`get_project_context`	`query`	`ContextEngine.gather` со scope `project`
`search_memory`	`query`	`ContextEngine.gather` со scopes `memory`, `other_chats`

Файловые инструменты работают только внутри `Project.root_path` действующего проекта: путь с `..` или симлинк наружу даёт ошибку. Действующий проект — это слот `project_id`, а если его нет, то `task.project_id`. Запуск тестов и Git появятся позже с `side_effects=True`, и для них восстановление работает иначе (раздел 8).

8. Персистентность, возобновление, конкурентность

Правило одно: сначала коммит, потом событие. Долгие операции (LLM, инструменты, `assess`) идут между транзакциями. Шаблон каждого шага: прочитать → посчитать вне транзакции → записать результат одной транзакцией с CAS по `version`.

Снапшоты. Каждый `tasks.update` сопровождается `snapshots.add` в той же транзакции (хелпер `_save`). Снапшот содержит статус, frame, счётчики и переход `(from, to, reason)`. По снапшотам можно воспроизвести историю задачи и объяснить, почему был задан вопрос.

Возобновление (`resume`).

1. Прочитать `Task`. Терминальная → вернуть как есть. `waiting_user` → повторно отдать `questions_asked` с открытыми вопросами.
2. **Починить переписку:** для каждого `tool_call` без соответствующего `tool_result` записать результат-ошибку, а висящие `ToolRun(started)` пометить `interrupted`. API Anthropic и OpenAI требуют результат на каждый tool call, иначе следующий вызов упадёт.
3. Текст ошибки зависит от `side_effects`: для read-only — «вызов прерван, можно повторить», для инструментов с побочным эффектом — «результат неизвестен, не повторяй без проверки».
4. Запустить `_drive` с текущего статуса. Для `analyzing` trigger восстанавливается так: `after_answer`, если у задачи есть ответы, иначе `initial`.

Идемпотентность. `client_message_id` записывается в `inbox` в той же транзакции, что и само сообщение. Повтор запроса (двойной клик, ретрай клиента) не создаст вторую задачу и не засчитает ответ дважды.

Конкурентность.

- В одном процессе задачу ведёт не больше одной корутины (`asyncio.Lock` на `task_id`).
- Между процессами защищает CAS по `version`. Проигравший получает `ConcurrencyError`, перечитывает задачу и останавливается.

- `cancel` не берёт lock: он делает CAS-запись `cancelled` (до 3 попыток). Ближайший коммит цикла проиграет и цикл завершится. Выполняющийся в этот момент вызов LLM доработает, но его результат не сохранится.
- На чат допускается одна активная задача. Сообщение во время `executing` получает `TaskBusy`. Очередь сообщений — возможное расширение после MVP.

9. Ошибки, ретраи, лимиты

Все лимиты собраны в `OrchestratorConfig`. Значения ниже — стартовые, их нужно подобрать на реальных сценариях.

Параметр	По умолчанию	Что происходит при превышении
`max_questions_per_round` (K)	3	лишние вопросы отбрасываются
`max_clarification_rounds`	3	`assess` не может вернуть ASK → PROCEED или `failed(clarification_exhausted)`
`max_agent_steps`	12	`failed(budget_exceeded)`
`max_tool_calls`	30	`failed(budget_exceeded)`
`max_repeated_tool_call`	2	вызов не выполняется, LLM получает ошибку
`max_tokens_per_task`	200 000	`failed(budget_exceeded)`
`context_token_budget`	8 000	Context Engine обрезает контекст
`max_tool_output_chars`	12 000	вывод обрезается с пометкой
`llm_retries` / `llm_timeout_s`	3 / 60 с	`failed(llm_error)`
`max_run_seconds`	600	`failed(budget_exceeded)`

Классы ошибок (`harness/contracts/errors.py`) и реакция на них:

Ошибка	Кто бросает	Реакция оркестратора
`LLMTransientError`, `asyncio.TimeoutError`	LLM-адаптер	повтор с экспоненциальной задержкой 1, 2, 4 с; после 3 неудач → `llm_error`
`LLMFatalError`	LLM-адаптер (неверный ключ, переполненный контекст)	сразу `failed(llm_error)`
ошибка аргументов или исключение в инструменте	Executor	`ToolResult(ok=False)` отдаётся модели, задача продолжается
нарушение контракта `Assessment`	оркестратор при проверке	исправление по правилам раздела 4 + warning в лог
любое исключение из `assess` / `parse_answers` / `gather`	модуль Родиона или Егора	`failed(internal_error)`
`ConcurrencyError`	Storage	перечитать задачу и остановить цикл
прочие ошибки Storage	Storage	пробросить наружу — транзакция откатилась, состояние не испорчено, поможет `resume`

При `failed` пользователь получает короткое объяснение: что не получилось и какие сведения так и не удалось получить (открытые блокирующие пробелы). Текст думания модели и стек-трейсы наружу не отдаются.

10. Сборка промпта и наблюдаемость

Финальный промпт для выполнения собирает `PromptBuilder` оркестратора. Context Engine отдаёт отобранные блоки, а Clarification Engine сам собирает промпты для `assess`. Порядок частей — от стабильных к меняющимся, чтобы работал кеш префикса у провайдера.

1. **System:** роль и правила. Не додумывать критичные параметры. При новой критичной неопределённости вызвать `request_clarification`. Ссылаться на контекст в виде `[ctx:<id>]`. Отвечать на языке пользователя.

2. **Карточка задачи** (user-сообщение): цель, слоты с источниками, допущения, исходный запрос.
3. **Контекст:** блоки в формате `[ctx:<id> | <kind>:<ref>]` + `summary` или `text`.
4. **Переписка задачи:** вопросы и ответы уточнения, затем пары `assistant(tool_calls)` → `tool(result)` в хронологическом порядке.

Переписка пока не сжимается: при `max_agent_steps = 12` и выводе до 12 000 символов на инструмент она помещается в контекст. Если перестанет помещаться, старые tool-результаты будем заменять на summary.

Наблюдаемость:

- **Логи** — структурные, через `logging` с `extra={"task_id", "status", "step"}`. Пишем каждый переход, каждый вызов LLM (токены, латентность) и каждое нарушение контракта.
- **Трасса** — из БД: `messages` + `tool_runs` + `task_snapshots` полностью восстанавливают, что происходило. `task.context_block_ids` объясняет, откуда взят контекст.
- **Метрики для оценки skill'а:** доля задач с уточнением, среднее число раундов и вопросов, доля вопросов с ответом `skipped`, распределение `failure.code`, число запусков `request_clarification` во время выполнения. Все считаются SQL-запросами по уже сохранённым таблицам.

11. Каркас кода, план и открытые вопросы

Каркас `harness_orchestrator` уже работает на заглушках: 38 тестов проходят, `demo.py` проигрывает сценарий «Проверь тесты проекта» с двумя проектами. Python 3.11+, единственная зависимость — pydantic 2.

Файл	Что внутри	Владелец
`harness/contracts/models.py`, `ports.py`, `errors.py`	модели, протоколы, исключения	все трое
`harness/orchestrator/orchestrator.py`	публичное API, `_drive`, `_analyze`, `_execute_step`, `resume`	Сергей
`harness/orchestrator/fsm.py`, `sanitize.py`, `prompt.py`, `config.py`	переходы, проверка `Assessment`, промпт, лимиты	Сергей

Файл	Что внутри	Владелец
`harness/tools/registry.py`, `builtin.py`	Executor с проверкой схемы и таймаутом, 5 инструментов	Сергей
`harness/fakes/memory_storage.py`, `simple_context.py`	эталон поведения Storage и Context Engine	образец для Егора
`harness/fakes/rule_clarifier.py`	движок на правилах + формула `priority`	образец для Родиона
`harness/fakes/scripted_llm.py`	LLM по сценарию, для тестов без сети	все
`harness/testing/storage_contract.py`	8 контрактных проверок Storage	Егор запускает на своей реализации
`harness/testing/clarifier_contract.py`	валидаторы + 6 сценариев	Родион запускает на своём движке

Что покрывают тесты оркестратора: сценарий без вопросов и с уточнением; отмена ответом и во время выполнения; новый запрос вместо ответа; исчерпание раундов без додумывания; уточнение во время выполнения; неверные аргументы и выход за пределы проекта; повторные вызовы; лимит шагов; ретраи и фатальные ошибки LLM; падение чужого модуля; дубликат `client_message_id`; `TaskBusy`; `resume` после падения; непрерывность версий и снапшотов.

План работ:

1. **Заморозить контракты v0.1.** Все трое читают `harness/contracts` и этот документ, закрывают открытые вопросы ниже, правки идут одним PR.
2. **Параллельная разработка на заглушках.** Егор — PostgreSQL-репозитории до зелёного `storage_contract`. Родион — движок на LLM до зелёного `clarifier_contract`. Сергей — LLM-адаптер к выбранному SDK и CLI-интерфейс.
3. **Интеграция.** Заменить заглушки реальными модулями в `Orchestrator(...)` и прогнать `tests/test_orchestrator.py` с реальным Storage. Логи `contract_violation` должны быть пустыми.

4. **Набор реальных сценариев.** 15–20 запросов с ожидаемыми вопросами (или их отсутствием), подбор лимитов и весов `priority`, метрики из раздела 10.
5. **После MVP.** HTTP + SSE на FastAPI, инструменты с побочными эффектами (запуск тестов, Git), сжатие переписки, переход на LangGraph только при усложнении ветвлений.

Открытые вопросы для команды:

- [ ] Кто владеет Context Engine? Предлагаю Егора, потому что движок работает поверх его таблиц.
- [ ] Tool Registry, LLM-адаптер и Interface — за Сергеем? Явно они не распределены.
- [ ] Добавляем статус `cancelled` в MVP-документ? Без него отмена превращается в `failed`, и метрики смешиваются.
- [ ] Какой LLM SDK? От него зависит только адаптер `LLMClient`.
- [ ] Всё асинхронно (`async`) — Егору нужен `asyncpg` или SQLAlchemy async. Всех устраивает?
- [ ] Сообщение во время `executing`: отказ `TaskBusy` (сейчас) или очередь?
- [ ] Новый запрос вместо ответа на вопрос: отменять старую задачу (сейчас) или спрашивать, что с ней делать?